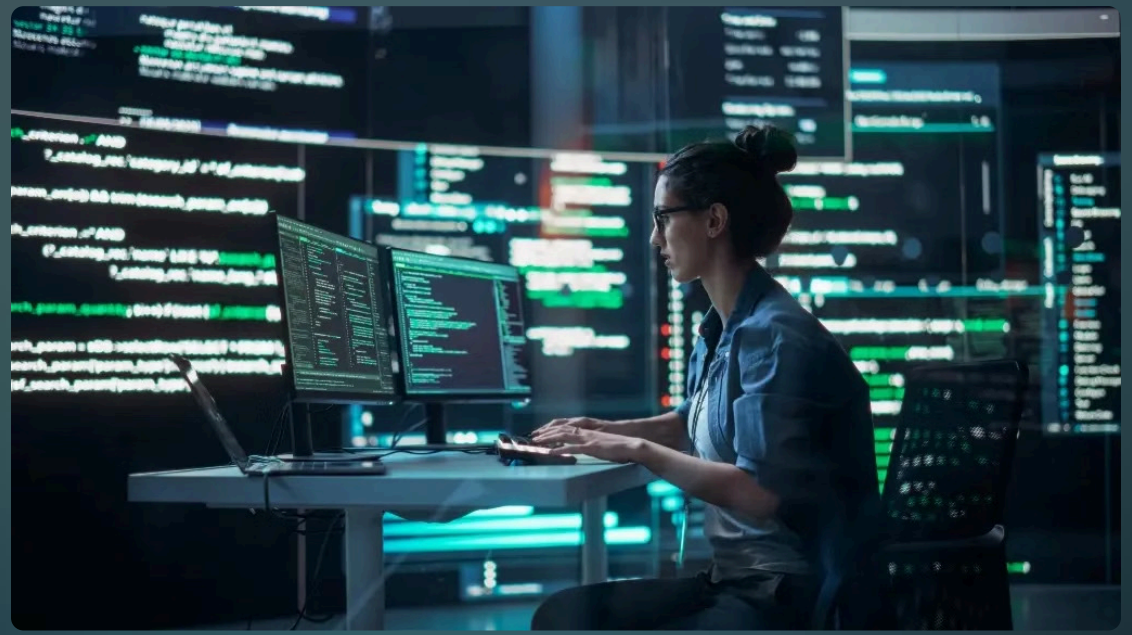
# Formação Completa

# AWS Developer Associate



# Arquiteturas e Padrões de Desenvolvimento.

## Parte 1

# Sistemas Stateful

**1** **Definição**

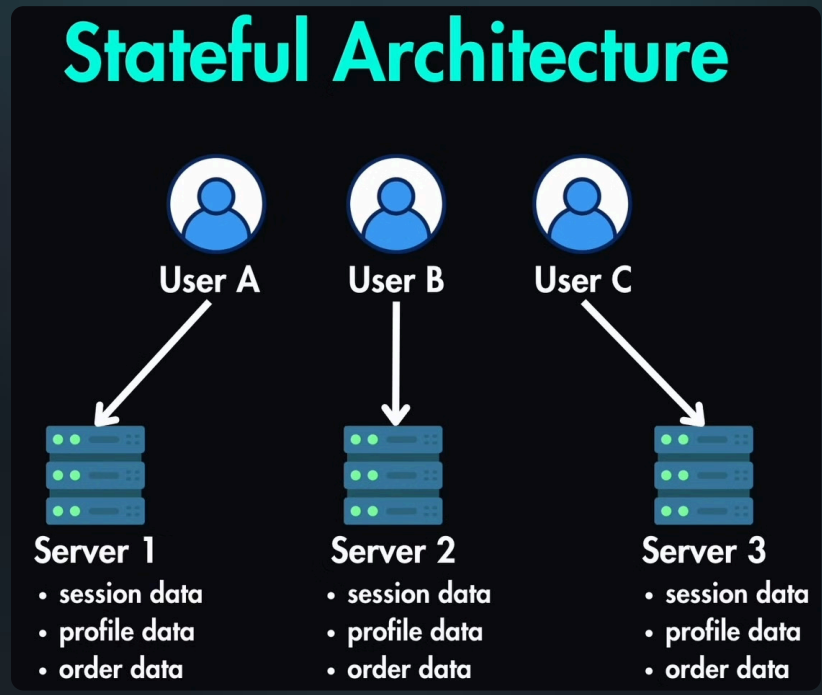
Mantém estado entre requisições.

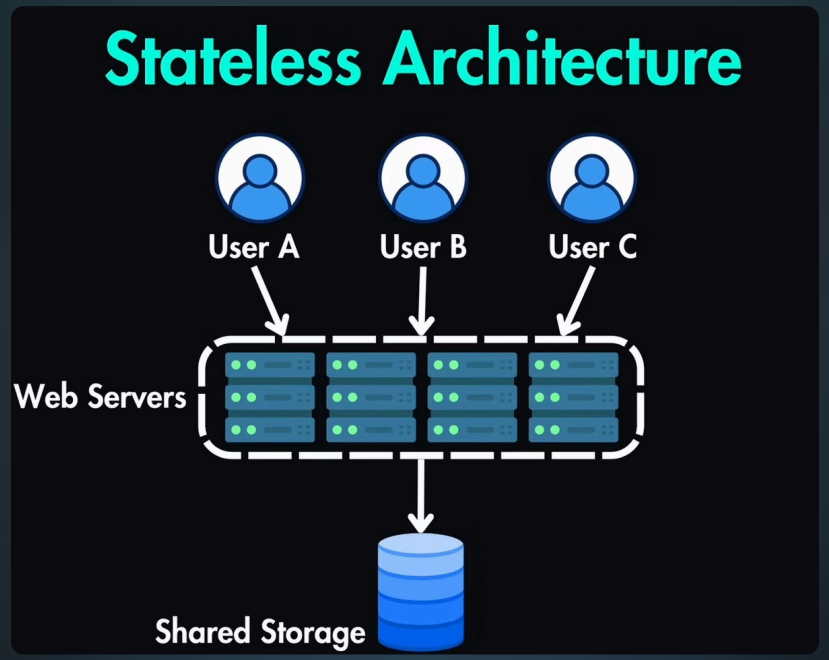
**2** **Armazenamento**

Guarda informações do usuário ou transação.

**3** **Continuidade**

Preserva contexto entre interações.






# Sistemas Stateless

### Definição

Cada requisição é independente.

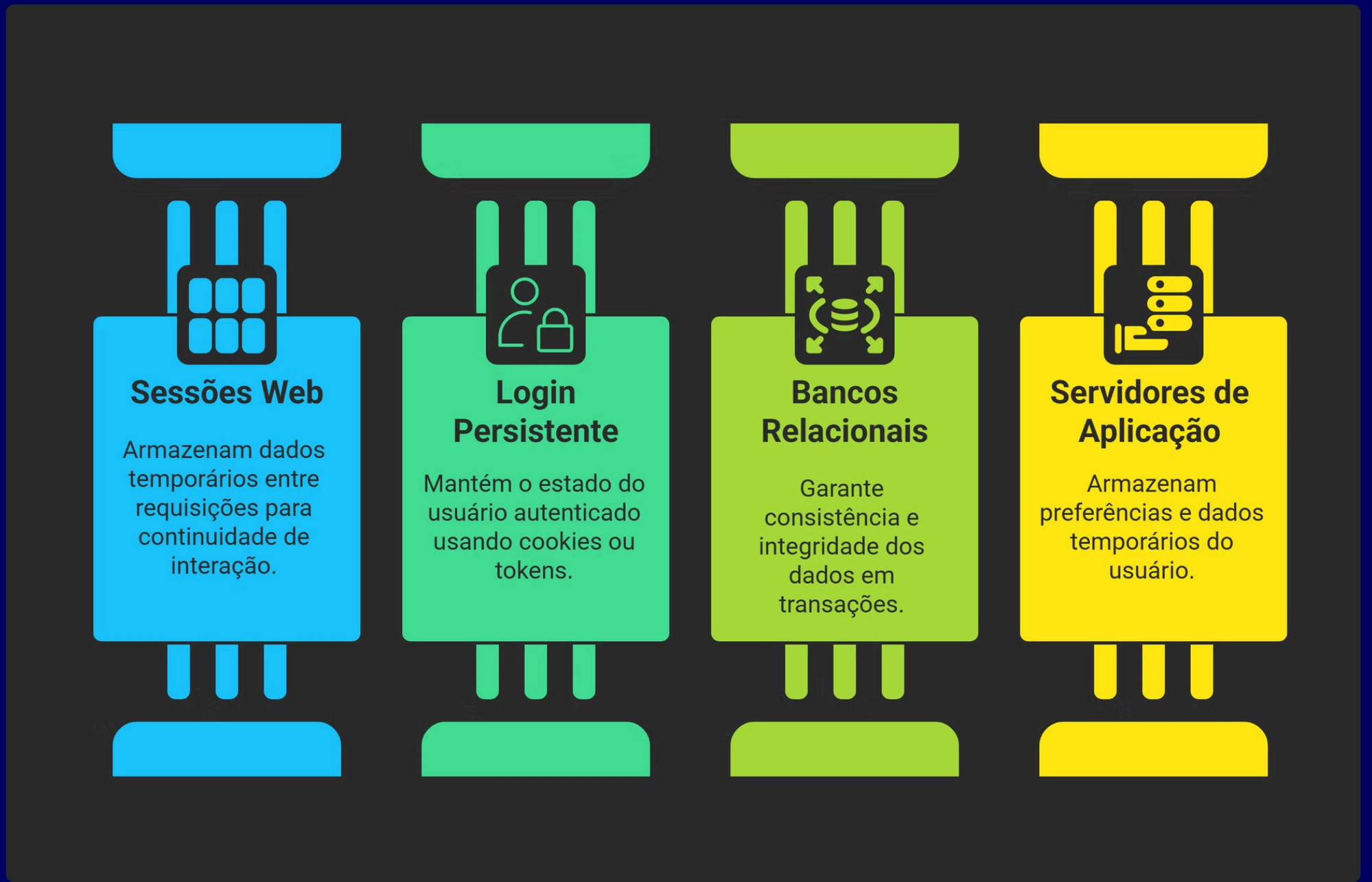
### Sem Memória

Não armazena informações persistentes.

### Autonomia

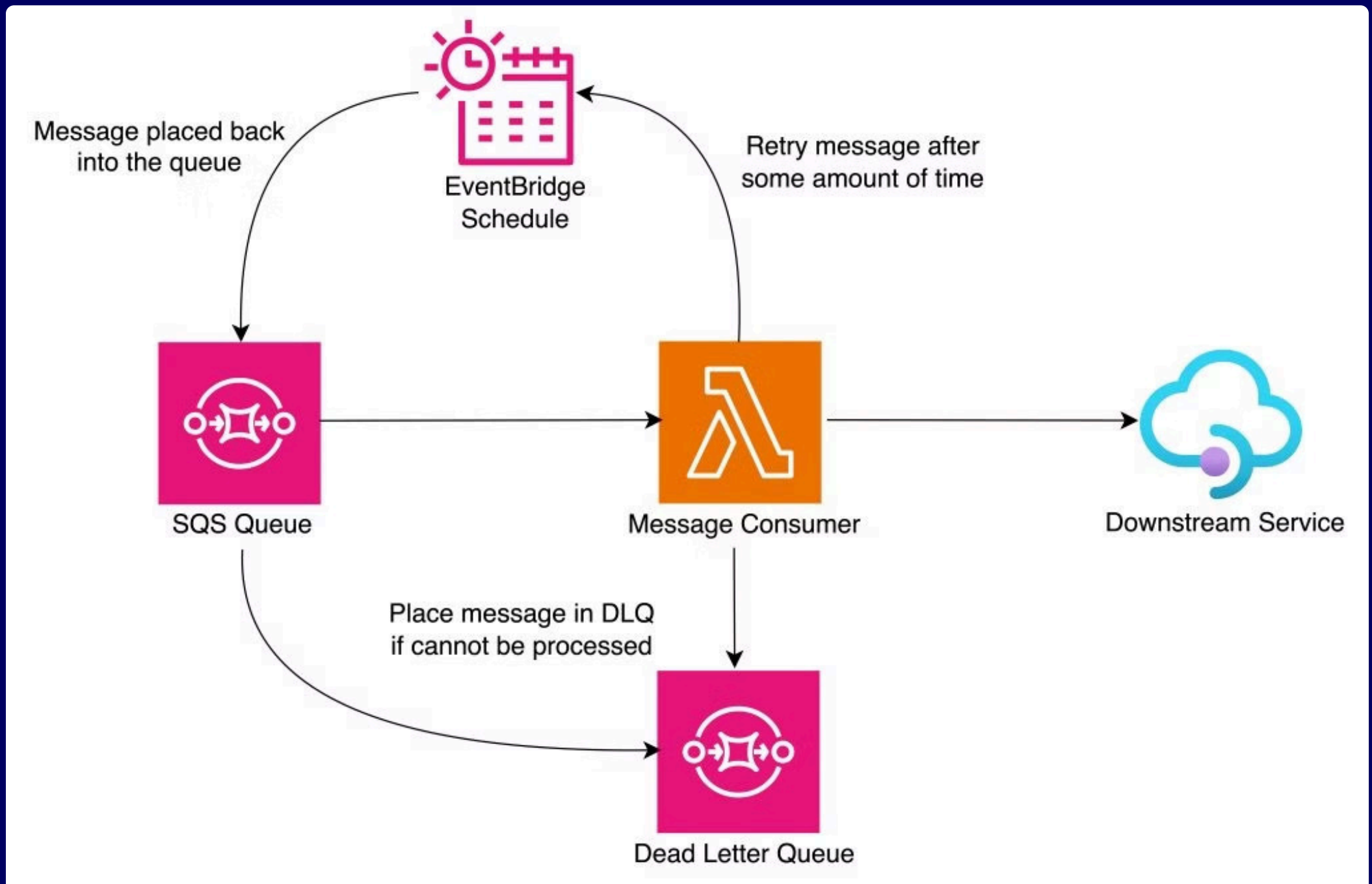
Toda requisição contém dados necessários.

# Exemplos Stateful



# Exemplos Stateless

Esse diagrama demonstra um padrão robusto e escalável para processamento assíncrono de mensagens. Ao usar serviços como SQS, Lambda e EventBridge, podemos criar sistemas que são mais resilientes, escaláveis e fáceis de manter.






# Benefícios: Stateful

1. **Personalização**

   Experiência adaptada ao usuário.

2. **Sessões Persistentes**

   Continuidade na navegação.

3. **Interação Contínua**

   Fluxos complexos mantêm contexto.

# Benefícios: Stateless

1

### Escalabilidade

Fácil adição de novos servidores.

2

### Tolerância a Falhas

Falha em um nó não afeta outros.

3

### Economia

Menor custo de infraestrutura.





# Idempotência: Conceito

### Definição

Múltiplas chamadas com mesmos parâmetros resultam no mesmo efeito.

### Proteção

Evita duplicidade de dados em operações reexecutadas automaticamente.

### Implementação

Use condições como attribute_not_exists para proteger a integridade dos dados.

# Operações Idempotentes



## GET

Utilizado para recuperar dados do servidor. Não altera o estado do servidor, garantindo resultados consistentes.



## PUT

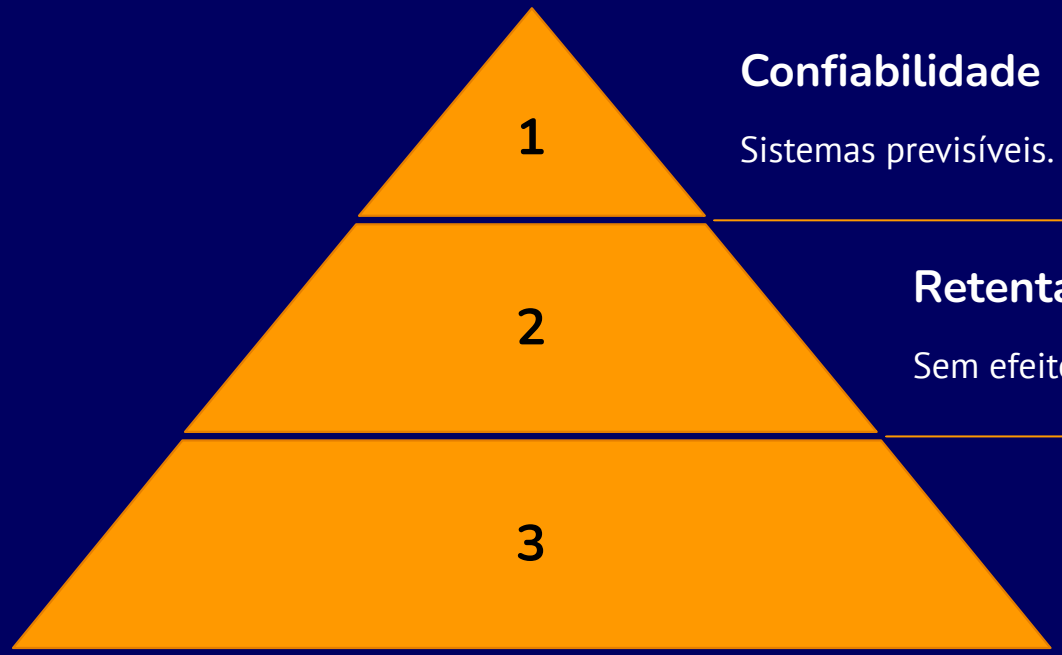
Usado para atualizar ou substituir um recurso existente com novos dados. Garante resultados consistentes ao enviar os mesmos dados várias vezes.



## DELETE

Empregado para remover recursos, podendo afetar múltiplos itens. Garante resultados consistentes ao deletar o mesmo recurso repetidamente.

# Importância da Idempotência




**Confiabilidade**

Sistemas previsíveis.

**Retentativas Seguras**

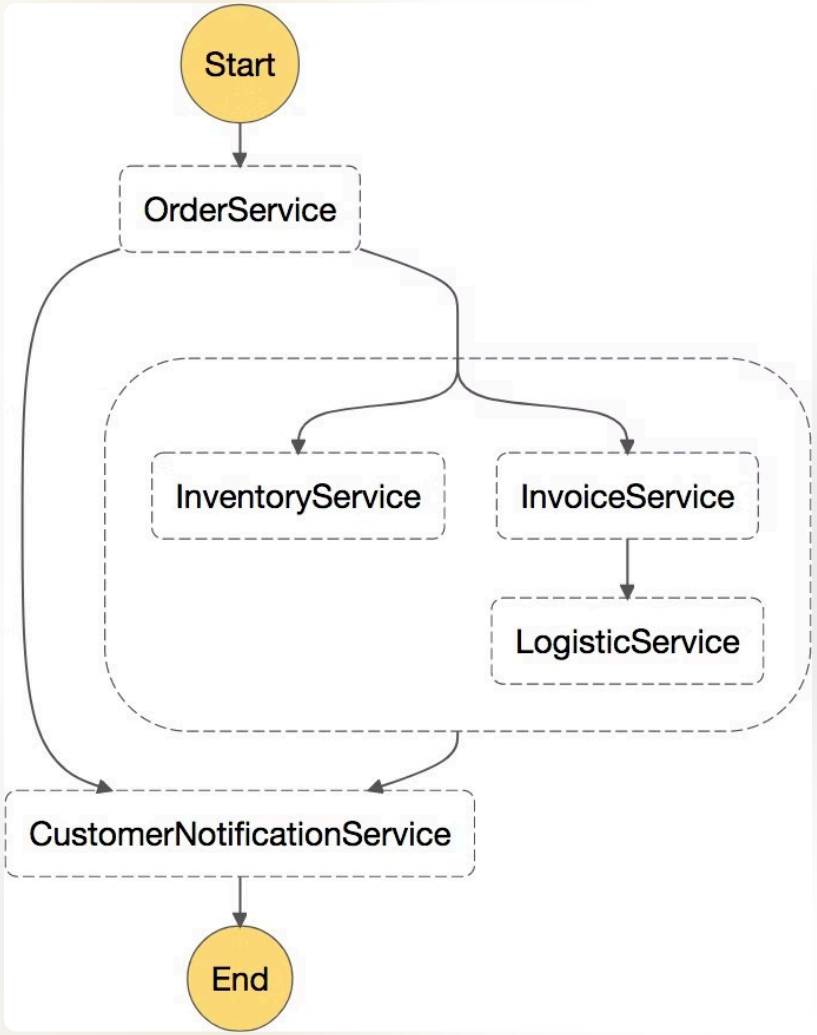
Sem efeitos colaterais.

**Transações Distribuídas**

Evita duplicidade.

# Comunicação Síncrona






# Comunicação Assíncrona

| Característica | Descrição |
| --- | --- |
| Processamento | Posterior, sem bloqueio |
| Desacoplamento | Emissor independente do receptor |
| Exemplo | Filas (SQS) ou mensagens (SNS) |



# Benefícios: Comunicação Síncrona

1

**Tempo Real**

Resposta imediata para usuário.

2

**Simplicidade**

Implementação direta.

3

**Consistência**

Dados sempre atualizados.

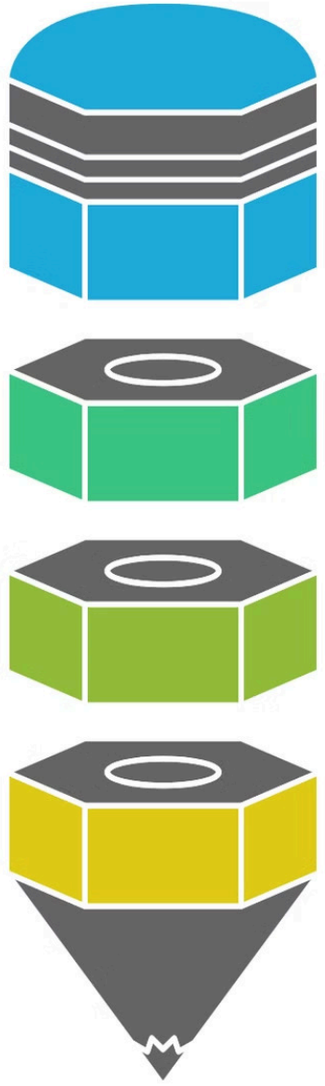
# Benefícios: Comunicação Assíncrona

## Comunicação Assíncrona

Permite a troca de mensagens sem a necessidade de simultaneidade

## Resiliência

Capaz de se recuperar de falhas rapidamente



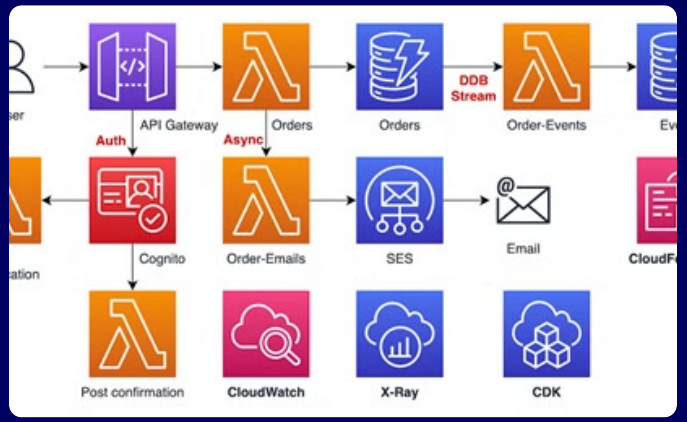
## Escalabilidade

Capaz de crescer e lidar com cargas de trabalho aumentadas

## Desacoplamento

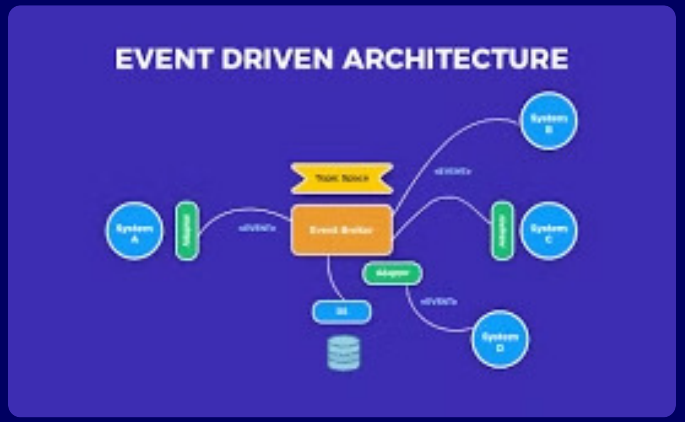
Componentes independentes para maior flexibilidade

# AWS Lambda: Conceito




## Serverless

Execução de código sob demanda.




## Event-driven

Acionado por eventos.

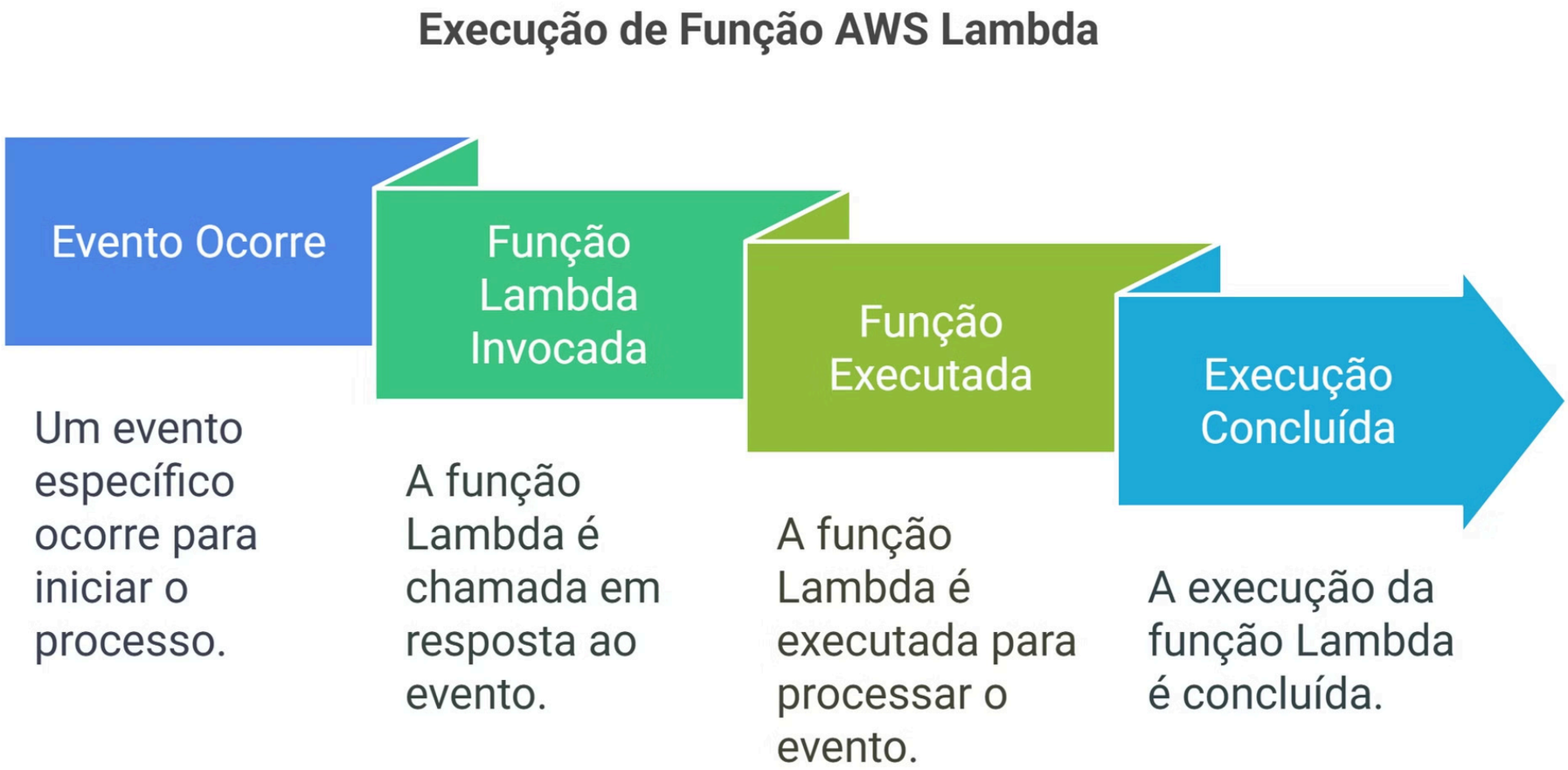


## Escalável

Ajusta-se conforme necessidade.

# AWS Lambda é Event-driven

AWS Lambda é **event-driven** porque sua execução é disparada por eventos, em vez de rodar continuamente como um servidor tradicional. Isso significa que a função Lambda só é executada quando ocorre um evento específico, o que a torna altamente escalável e eficiente em termos de custo.



# Benefícios do AWS Lambda

## Custo

Pague apenas pelo tempo de execução.

## Escalabilidade

Automática conforme demanda.

## Integração

Conecta-se a diversos serviços AWS.

# Exemplos de Uso: Lambda

1. **Processamento de Imagens**

   Redimensiona fotos enviadas ao S3.

2. **Workflows**

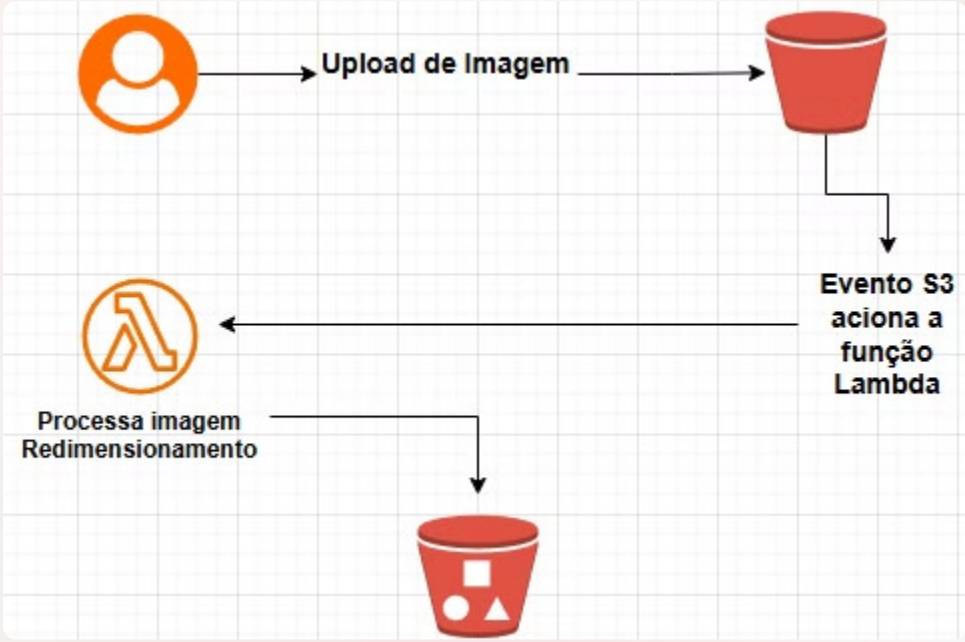
   Orquestra processos baseados em eventos.

3. **ETL**

   Transforma dados entre sistemas.

4. **APIs**

   Backend serverless para aplicações.





# Amazon SNS: Conceito

### Notificação

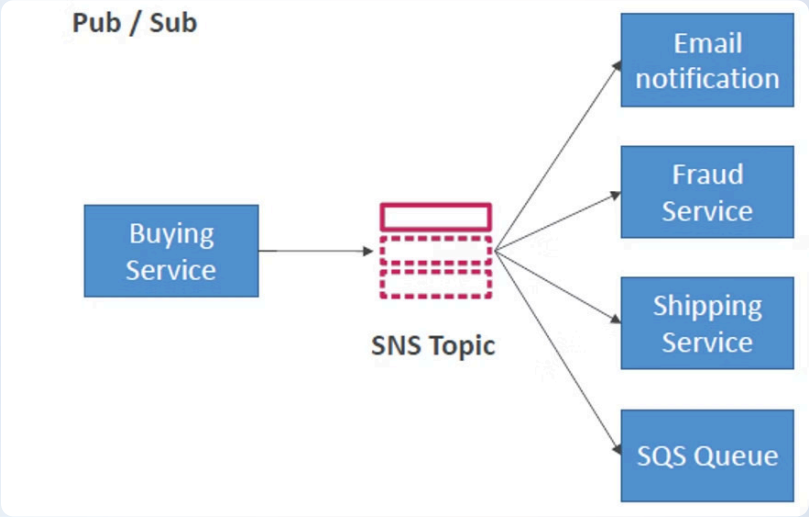
Serviço de mensagens push.

### Publicação

Envia mensagens para tópicos.

### Assinatura

Recebe mensagens conforme interesse.




# Modelo Pub/Sub

1

## Publicadores

Enviam mensagens para tópicos.

2

## Tópicos

Canais de distribuição de mensagens.

3

## Assinantes

Recebem mensagens de interesse.

# Benefícios do Amazon SNS

Visão Geral dos Benefícios do Amazon SNS

**Filtragem de Mensagens**

Garante que os assinantes recebam apenas conteúdo relevante

**Flexibilidade de Entrega de Protocolos**

Suporta vários métodos de entrega de mensagens



**Comunicação Escalável**

Facilita a interação entre serviços em escala

**Baixa Latência**

Assegura entrega rápida de mensagens

# Exemplos de Uso: SNS

Métodos de Comunicação



**Notificações Móveis**

Notificações enviadas para dispositivos móveis.



**Gatilhos Lambda**

Aciona funções serverless automaticamente.



**Alertas**

Notificações para equipes sobre eventos críticos.



**Emails**

Envio de comunicações em massa para usuários.

# Amazon SQS: Conceito

### Fila

Armazena mensagens para processamento.

### Assíncrono

Desacopla produtores e consumidores.

### Durável

Mensagens persistem até processamento.



# Funcionamento do SQS

**1** **Envio**

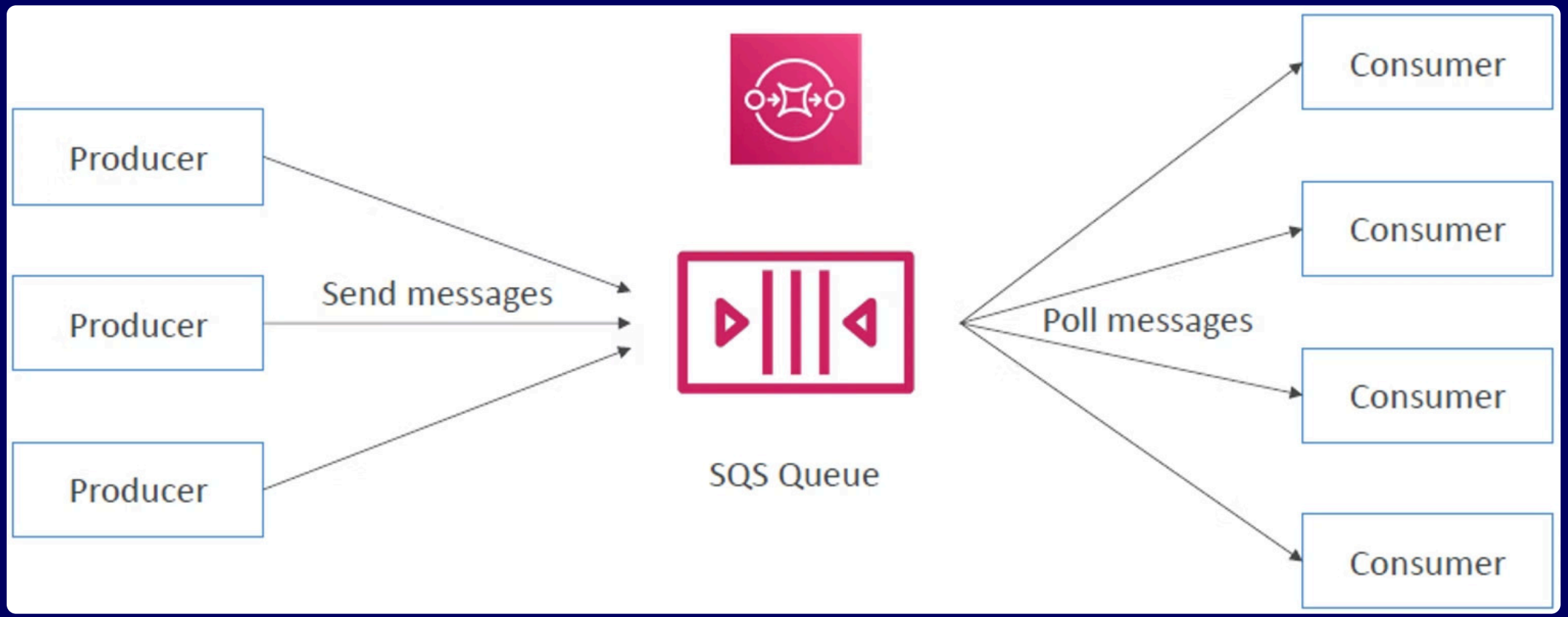
Produtor coloca mensagem na fila.

**2** **Armazenamento**

Mensagem fica disponível para consumo.

**3** **Consumo**

Consumidor processa e remove da fila.



# Filas Standard vs FIFO

## Standard

Alto throughput

Ordem não garantida

Possível duplicação

## FIFO

Ordem garantida

Processamento único

Throughput limitado

# Benefícios do Amazon SQS




**Resiliência**

Alta disponibilidade.

**Desacoplamento**

Independência entre serviços.

**Escalabilidade**

Gerencia picos de demanda.

**Confiabilidade**

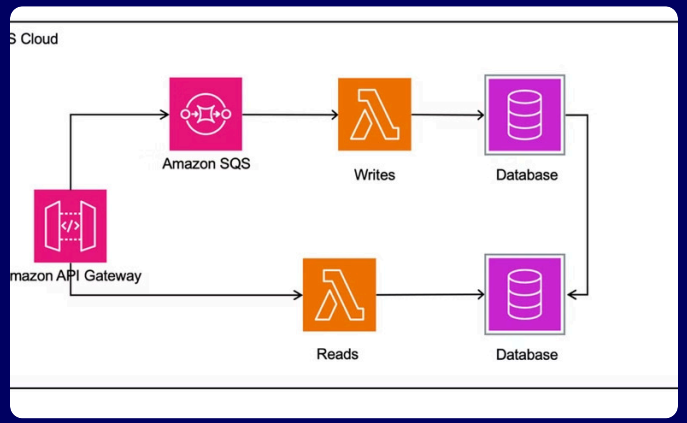
Garante processamento de mensagens.

# Exemplos de Uso: SQS



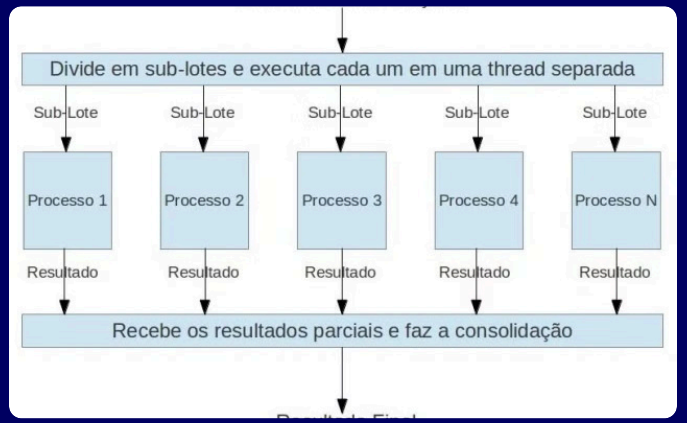

## E-commerce

Processamento de pedidos.




## Microserviços

Comunicação desacoplada.




## Processamento em Lote

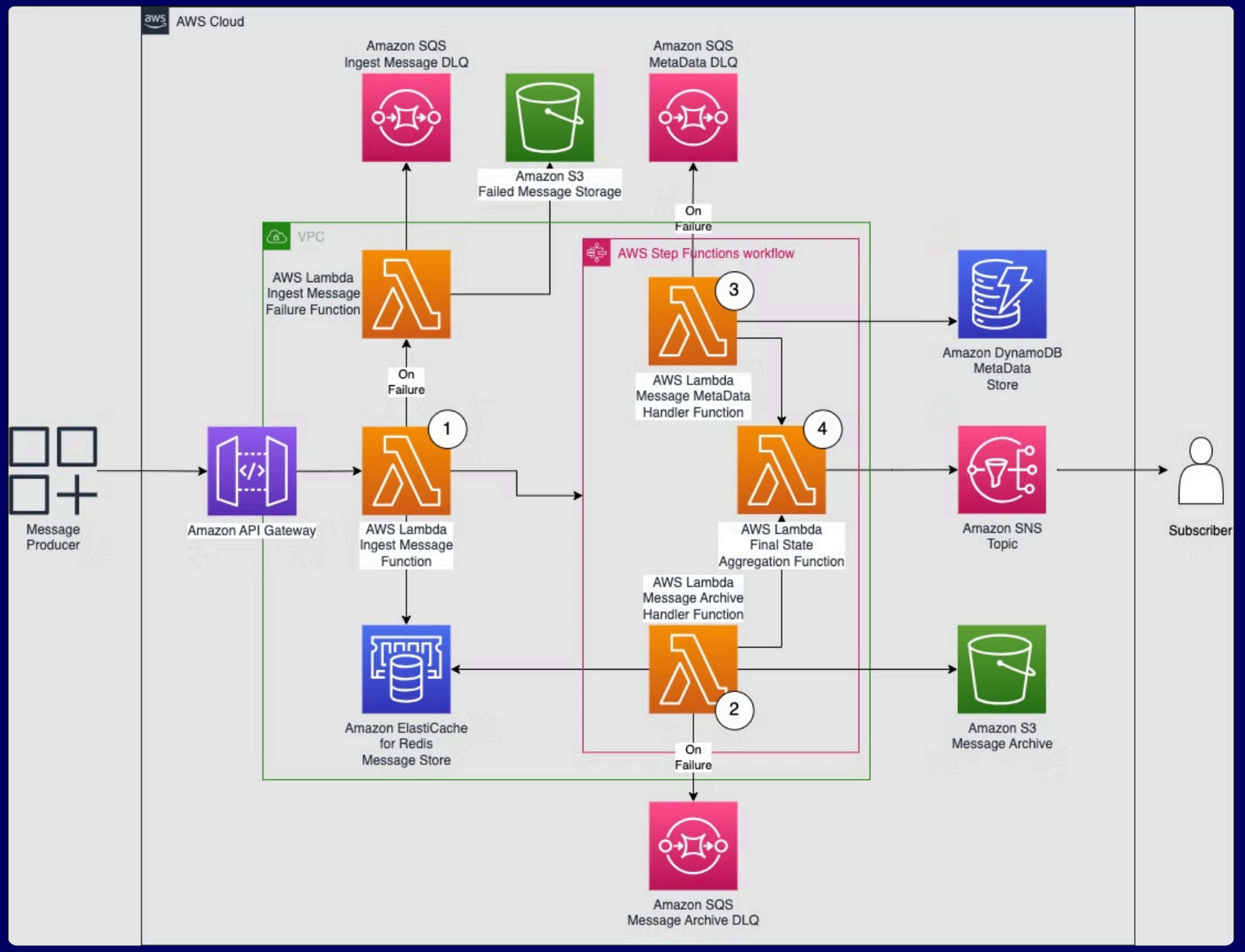
Gerencia trabalhos em background.

# Comparativo: SNS vs SQS

| Característica | SNS | SQS |
|---|---|---|
| Modelo | Pub/Sub | Fila |
| Entrega | Push | Pull |
| Destinatários | Múltiplos | Único |



# Integrando Serviços AWS




Combinação de serviços para arquiteturas robustas.

# Resumo dos Conceitos

### Stateful vs Stateless

Gerenciamento de estado em sistemas.

### Idempotência

Operações repetíveis com mesmo resultado.

### Síncrono vs Assíncrono

Modelos de comunicação entre serviços.

# Serviços AWS Abordados



## Lambda

Computação serverless event-driven.



## SNS

Notificações pub/sub.



## SQS

Filas de mensagens.

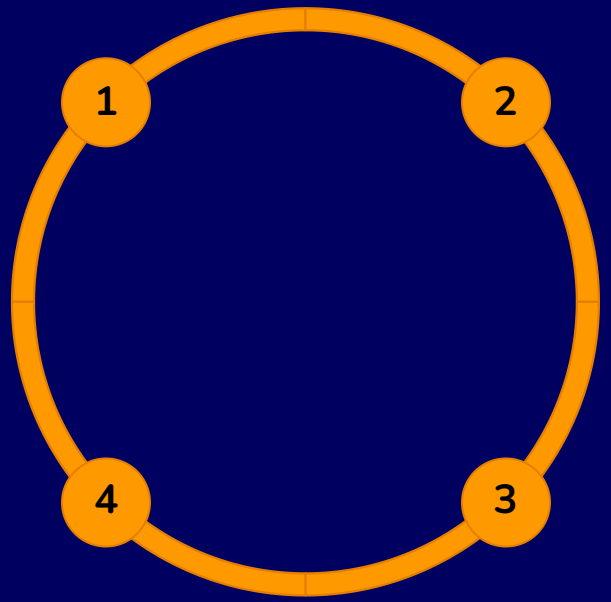
# Importância na Arquitetura

**Escalabilidade**

Suporta crescimento do sistema.

**Manutenibilidade**

Facilita evolução do sistema.




**Resiliência**

Tolerância a falhas.

**Eficiência**

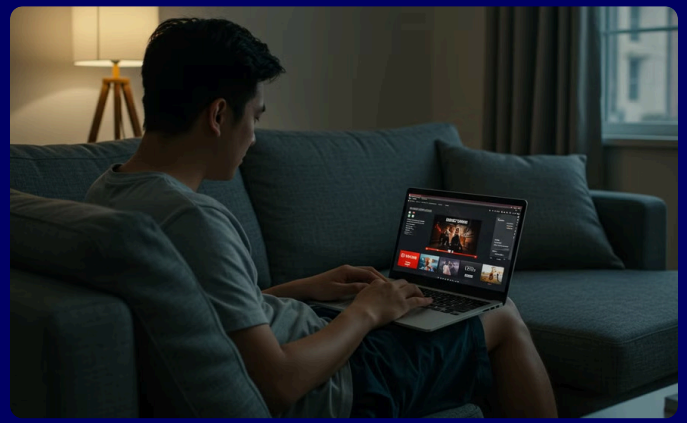
Otimização de recursos.

# Casos de Uso Reais



## E-commerce

Processamento de pedidos com SQS.



## Streaming

Processamento de mídia com Lambda.




## Fintech

Notificações financeiras com SNS.